OSSP

Skill Week - 1

Roll no - 2520090085 Name-: Vakkalanka Sai Bhaskara Sujit

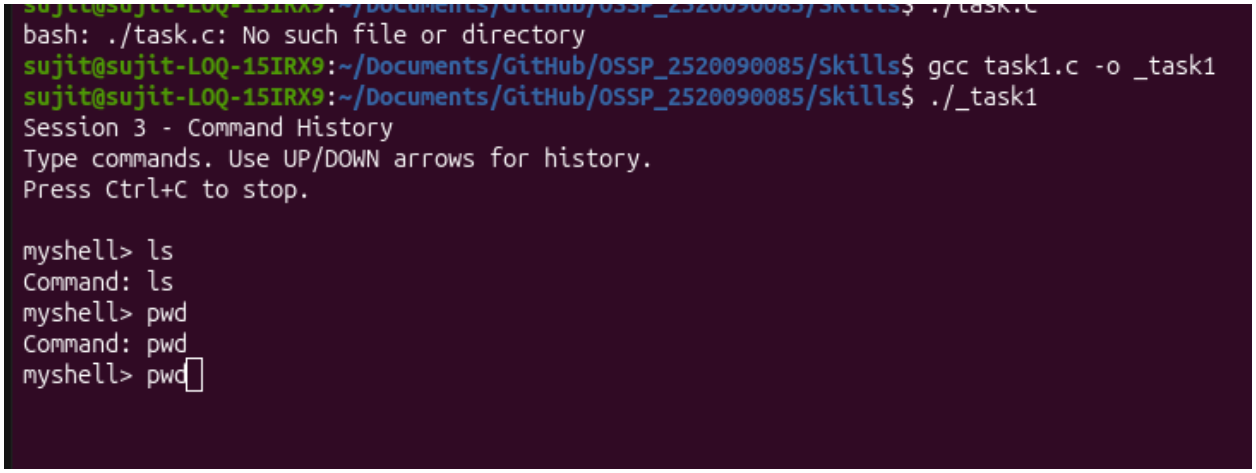
Task 1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

```
 GNU nano 8.7.1                                                task1.c *
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_HISTORY 100
#define BUFFER_SIZE 100

char history[MAX_HISTORY][BUFFER_SIZE];
int history_count = 0;
int history_index = 0;

void enable_raw_mode(struct termios *old) {
    struct termios raw;
    tcgetattr(STDIN_FILENO, old);
    raw = *old;
    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void restore_mode(struct termios *old) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, old);
}

void save_command(char *command) {
    if (strlen(command) == 0) return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++) {
            strcpy(history[i - 1], history[i]);
        }
        strcpy(history[MAX_HISTORY - 1], command);
    }
    history_index = history_count;
}

int main() {
    struct termios old;
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    printf("Session 3 - Command History\n");
    printf("Type commands. Use UP/DOWN arrows for history.\n");
    printf("Press Ctrl+C to stop.\n\n");

    enable_raw_mode(&old);
```

```
 GNU nano 8.7.1
S
    enable_raw_mode(&old);

    while (1) {
        index = 0;
        buffer[0] = '\0';
        printf("myshell> ");
        fflush(stdout);

        while (1) {
            ch = getchar();

            // Enter key (Carriage return / Newline)
            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';
                printf("\n");

                if (strcmp(buffer, "exit") == 0) {
                    restore_mode(&old);
                    printf("Exiting...\n");
                    return 0;
                }

                if (strlen(buffer) > 0) {
                    printf("Command: %s\n", buffer);
                    save_command(buffer);
                }
                break;
            }
            // Backspace key
            else if (ch == 127 || ch == 8) {
                if (index > 0) {
                    index--;
                    buffer[index] = '\0';
                    printf("\b \b");
                    fflush(stdout);
                }
            }
            // Escape sequences (Arrow keys)
            else if (ch == 27) {
                char ch2 = getchar();
                if (ch2 == '[') {
                    char ch3 = getchar();
                    // UP arrow
                    if (ch3 == 'A') {
                        if (history_index > 0) {
                            history_index--;
                            while (index > 0) {
                                printf("\b \b");
                                index--;
                            }
```

```
                    }
                    // DOWN arrow
                    else if (ch3 == 'B') {
                        if (history_index < history_count - 1) {
                            history_index++;
                            while (index > 0) {
                                printf("\b \b");
                                index--;
                            }
                            strcpy(buffer, history[history_index]);
                            index = strlen(buffer);
                            printf("%s", buffer);
                            fflush(stdout);
                        } else {
                            history_index = history_count;
                            while (index > 0) {
                                printf("\b \b");
                                index--;
                            }
                            buffer[0] = '\0';
                        }
                    }
                }
            }
            // Normal printable character
            else if (index < BUFFER_SIZE - 1) {
                buffer[index++] = ch;
                buffer[index] = '\0';
                putchar(ch);
                fflush(stdout);
            }
        }
    }

    restore_mode(&old);
    return 0;
}
```

Output-
:



Task2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    char *buffer;
    size_t size = INITIAL_SIZE;
    size_t length = 0;
    int ch;

    Node *head = NULL;
    Node *tail = NULL;

    buffer = malloc(size);
    if (buffer == NULL) {
        perror("malloc failed");
        return 1;
    }

    printf("Session 3 - Dynamic Memory Management\n");
    printf("Enter commands. Type 'exit' to finish.\n\n");

    while (1) {
        length = 0;
        printf("myshell> ");
        fflush(stdout);

        while ((ch = getchar()) != '\n' && ch != EOF) {
            if (length + 1 >= size) {
                size *= 2;
                char *temp = realloc(buffer, size);
                if (temp == NULL) {
                    perror("realloc failed");
                    free(buffer);
                    return 1;
                }
                buffer = temp;
            }
            buffer[length++] = ch;
        }
        buffer[length] = '\0';

        if (strcmp(buffer, "exit") == 0) {
            break;
```

```
            break;
        }

        if (length == 0) {
            continue;
        }

        Node *new_node = malloc(sizeof(Node));
        if (new_node == NULL) {
            perror("malloc failed");
            free(buffer);
            return 1;
        }

        new_node->command = malloc(length + 1);
        if (new_node->command == NULL) {
            perror("malloc failed");
            free(new_node);
            free(buffer);
            return 1;
        }

        strcpy(new_node->command, buffer);
        new_node->next = NULL;

        if (head == NULL) {
            head = new_node;
            tail = new_node;
        } else {
            tail->next = new_node;
            tail = new_node;
        }

        printf("Stored command: %s\n", buffer);
        printf("Buffer size: %zu bytes\n", size);
    }

    free(buffer);

    Node *current = head;
    while (current != NULL) {
        Node *next = current->next;
        free(current->command);
        free(current);
        current = next;
    }

    printf("\nAll dynamically allocated memory released.\n");
    return 0;
}
s
```

Output-:

```
Enter commands. Type 'exit' to finish.

myshell> ls
Stored command: ls
Buffer size: 10 bytes
myshell> Indian
Stored command: Indian
Buffer size: 10 bytes
myshell> loq
Stored command: loq
Buffer size: 10 bytes
myshell> ^[[3~
```

Task : To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

GNU nano 8.7.1 task

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKEN_LEN 80
#define MAX_TOKENS 50

typedef struct {
    char value[MAX_TOKEN_LEN];
} Token;

int main() {
    char input[500];
    Token tokens[MAX_TOKENS];
    int token_count = 0;

    printf("Session 4 - Tokenization\n");
    printf("Enter a command: ");
    if (!fgets(input, sizeof(input), stdin)) return 0;

    int i = 0;
    while (input[i] != '\0' && input[i] != '\n') {
        // Skip leading whitespace
        while (isspace(input[i]) && input[i] != '\n' && input[i] != '\0') {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') break;

        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }

        int j = 0;
        if (input[i] != '|' && !isspace(input[i])) {
            while (input[i] != '\0' && !isspace(input[i]) && input[i] != '|' && j < MAX_TOKEN_LEN - 1) {
                tokens[token_count].value[j++] = input[i++];
            }
            tokens[token_count].value[j] = '\0';
            token_count++;
        } else if (input[i] == '|') {
            tokens[token_count].value[0] = '|';
            tokens[token_count].value[1] = '\0';
            token_count++;
            i++;
        }
    }

    printf("\nTokens identified:\n");
```

```
 GNU nano 8.7.1                                                                                                  task

        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }

        int j = 0;
        if (input[i] != '|' && !isspace(input[i])) {
            while (input[i] != '\0' && !isspace(input[i]) && input[i] != '|' && j < MAX_TOKEN_LEN - 1) {
                tokens[token_count].value[j++] = input[i++];
            }
            tokens[token_count].value[j] = '\0';
            token_count++;
        } else if (input[i] == '|') {
            tokens[token_count].value[0] = '|';
            tokens[token_count].value[1] = '\0';
            token_count++;
            i++;
        }
    }

    printf("\nTokens identified:\n");
    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    for (int k = 0; k < token_count; k++) {
        printf("Token %d: [%s]\n", k + 1, tokens[k].value);
    }

    printf("\nToken stream validation:\n");
    int valid = 1;
    for (int k = 0; k < token_count; k++) {
        if (strcmp(tokens[k].value, "|") == 0) {
            if (k == 0 || k == token_count - 1 || strcmp(tokens[k + 1].value, "|") == 0) {
                valid = 0;
                break;
            }
        }
    }

    if (valid) {
        printf("Token stream is valid.\n");
    } else {
        printf("Token stream is invalid.\n");
    }

    return 0;
}
```

Output-:

```
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano task3.c
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc task3.c -o task3
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./task3
Session 4 - Tokenization
Enter a command: Hello /World

Tokens identified:
Token 1: [Hello]
Token 2: [/World]

Token stream validation:
Token stream is valid.
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$
```

Task : To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50

int main() {
    char input[500];
    char *tokens[MAX_TOKENS];
    int token_count = 0;
    int valid = 1;

    printf("Session 4 - Parser\n");
    printf("Enter a command: ");
    if (!fgets(input, sizeof(input), stdin)) return 0;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command detected.\n");
        return 0;
    }

    char *token = strtok(input, " \t");
    while (token != NULL && token_count < MAX_TOKENS) {
        tokens[token_count++] = token;
        token = strtok(NULL, " \t");
    }

    printf("\nParse Structure:\n");
    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    // Syntax validation
    for (int i = 0; i < token_count; i++) {
        if (strcmp(tokens[i], "|") == 0) {
            if (i == 0 || i == token_count - 1) {
                valid = 0;
                printf("Syntax Error: Invalid pipe position.\n");
            }
            if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
                valid = 0;
                printf("Syntax Error: Consecutive pipes detected.\n");
            }
        }
    }

    if (!valid) {
        printf("\nParsing failed.\n");
        return 1;
```

```
GNU nano 8.7.1
    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    // Syntax validation
    for (int i = 0; i < token_count; i++) {
        if (strcmp(tokens[i], "|") == 0) {
            if (i == 0 || i == token_count - 1) {
                valid = 0;
                printf("Syntax Error: Invalid pipe position.\n");
            }
            if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
                valid = 0;
                printf("Syntax Error: Consecutive pipes detected.\n");
            }
        }
    }

    if (!valid) {
        printf("\nParsing failed.\n");
        return 1;
    }

    printf("Command: %s\n", tokens[0]);
    if (token_count > 1) {
        printf("Arguments:\n");
        for (int i = 1; i < token_count; i++) {
            if (strcmp(tokens[i], "|") != 0) {
                printf("  Argument %d: %s\n", i, tokens[i]);
            }
        }
    }

    printf("\nParse tree / execution structure:\n");
    printf("ROOT\n");
    printf(" └── COMMAND: %s\n", tokens[0]);
    for (int i = 1; i < token_count; i++) {
        if (strcmp(tokens[i], "|") == 0) {
            printf("      └── PIPE\n");
        } else {
            printf("      └── ARGUMENT: %s\n", tokens[i]);
        }
    }

    printf("\nSyntax validation: SUCCESS\n");
    printf("Execution structure generated successfully.\n");

    return 0;
}
```

Output-:

```
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ nano task4.c
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ gcc task4.c -o task4
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$ ./task4
Session 4 - Parser
Enter a command: ls -l | grep txt

Parse Structure:
Command: ls
Arguments:
  Argument 1: -l
  Argument 3: grep
  Argument 4: txt

Parse tree / execution structure:
ROOT
 └── COMMAND: ls
      └── ARGUMENT: -l
      └── PIPE
      └── ARGUMENT: grep
      └── ARGUMENT: txt

Syntax validation: SUCCESS
Execution structure generated successfully.
sujit@sujit-LOQ-15IRX9:~/Documents/GitHub/OSSP_2520090085/Skills$
```